

Birzeit University

Faculty of Engineering and Technology
Electrical and Computer Engineering Department

ENC55140 — Real-Time and Interfacing Techniques Laboratory

Course Project Brief

Smart Egg Incubator

An Automated Poultry-Hatching System Built with FreeRTOS on ESP32

with Real-Time Monitoring and Control via the Blynk IoT Platform

Course	ENC55140 – Real-Time Systems and Interfacing Techniques Lab
Project Duration	15 days —
Semester / Year	_____
Instructor	_____
Group No. / Members	_____
Submission Date	_____

Contents

1. Introduction and Motivation	3
2. Learning Objectives	3
3. Project Overview	3
4. Functional Requirements	4
5. Non-Functional and Real-Time Requirements	4
6. System Architecture	5
7. Hardware Requirements (Bill of Materials)	5
7.1 Safety Notes	6
8. Software and Development Tools	6
9. FreeRTOS Task Design	7
9.1 Inter-Task Communication and Synchronization	7
10. Blynk Dashboard Design	8
11. Safety and Fault Handling	8
12. Suggested Timeline	9
13. Deliverables	9
14. Suggested Grading Rubric	9
15. Expected Outcome / Prototype Description	9
16. Optional Extensions	10
17. References	10
Appendix A. Suggested FreeRTOS + Blynk Code Skeleton	11
Appendix B. Egg Incubation Parameter Reference	14

1. Introduction and Motivation

Artificial egg incubation is a classic real-world control problem: a small enclosure must be held at a precise temperature and humidity for roughly three weeks, while an egg tray is turned on a fixed schedule, all without a human present around the clock. It is an excellent vehicle for a Real-Time Systems and Interfacing Techniques project because the system genuinely has multiple concurrent, timing-critical activities — sensing, closed-loop actuation, a periodic mechanical action, a safety watchdog, and a network/IoT link — that must be designed as separate, correctly synchronized FreeRTOS tasks rather than as one long Arduino loop().

This document is a project brief for a “Smart Egg Incubator” built around an ESP32 microcontroller running FreeRTOS, with local sensing and actuation, and remote monitoring/control through the Blynk IoT platform (the same platform introduced in the course’s ESP32 + Blynk supporting document). It gives students the requirements, hardware list, real-time task design, IoT dashboard design, a suggested timeline, and a grading rubric needed to plan, build, and demonstrate a working prototype.

2. Learning Objectives

By completing this project, students should be able to:

- Decompose a real embedded control problem into independent, well-scoped FreeRTOS tasks with justified periods and priorities.
- Use core RTOS primitives — queues, mutexes/semaphores, and software timers — for safe inter-task communication and resource sharing.
- Interface digital/analog sensors (temperature, humidity) and actuators (relay-driven heater, humidifier, servo/motor) to the ESP32 through GPIO, PWM, and I2C/one-wire peripherals.
- Implement closed-loop (bang-bang or PID) control of a physical quantity against a real-time deadline.
- Integrate an IoT dashboard (Blynk) for live telemetry, remote override, and alerting without compromising the real-time behaviour of the local control loops.
- Apply basic safety/fault-tolerance practices — watchdogs, sensor-fault detection, fail-safe actuator states — to a system that controls a physical heating element.

3. Project Overview

Students will design and build an insulated incubator box that automatically:

- Measures internal temperature and relative humidity continuously.
- Keeps temperature within a safe band around 37.5 °C using a relay/SSR-driven heating element.
- Keeps relative humidity around 50–55% during the setting stage and raises it to 65–70% for the final “lockdown” days, using a small humidifier or fan-over-water arrangement.
- Automatically tilts/turns the egg tray several times a day and stops turning during lockdown, using a servo or small geared motor.
- Tracks the incubation day count and shows it locally (LCD/OLED) and remotely (Blynk app).
- Streams live readings to a Blynk dashboard and pushes an alert if temperature or humidity drifts out of range, or a sensor/actuator fault is detected.
- Allows a remote user to view status and, in a clearly marked “test/manual” mode, override the heater, humidifier, and turning motor from the app.

Reference incubation parameters used throughout this brief (chicken eggs, forced-air incubator): temperature $\approx$ 37.2–37.8 °C (99–100 °F); relative humidity 50–55% on days 1–17, raised to 65–70% on days 18–21 (lockdown); turning

≥ 3–5 times/day on days 1–17, turning stopped from day 18; total incubation ≈ 21 days. Duck (≈28 days), quail (≈17–18 days) and goose (≈28–35 days) eggs follow the same pattern with different setpoints and durations — teams targeting a species other than chicken should confirm exact figures with the instructor or a poultry-extension reference before calibrating the setpoints in Section 10.

4. Functional Requirements

The prototype shall satisfy the following functional requirements (FR). Teams may propose additional requirements but must implement at least FR1–FR8.

ID	Requirement
FR1	Measure internal temperature and relative humidity at least once per second.
FR2	Maintain temperature within the target band (default 37.2–37.8 °C) by switching a heating element via relay/SSR.
FR3	Maintain relative humidity at the setting-stage target (default 50–55%) and switch to the lockdown target (default 65–70%) from the configured lockdown day.
FR4	Automatically turn/tilt the egg tray at least 4 times per day, evenly spaced, and automatically stop turning from the lockdown day onward.
FR5	Track and display elapsed incubation days and days remaining until expected hatch (default 21 days, configurable).
FR6	Publish live temperature, humidity, heater/humidifier/motor state, and day counter to the Blynk dashboard at least every 2 seconds.
FR7	Send a Blynk push notification and raise a local audible/visual alarm if temperature or humidity leaves the safe range for longer than a configurable grace period (default 2 minutes).
FR8	Continue local closed-loop control and local display/alarm even if Wi-Fi or the Blynk connection is lost.
FR9	Provide a manual/test mode, switchable from the Blynk app, that lets the user directly command the heater, humidifier, and turning motor for calibration and demonstration.
FR10	Detect an implausible or missing sensor reading (e.g., NaN from the DHT22) and treat it as a fault rather than a valid control input.

5. Non-Functional and Real-Time Requirements

These requirements are what an RTOS-focused project is actually graded on: not just “does the incubator work” but “is the concurrency and timing design correct and justified.”

ID	Requirement
NFR1	The sensor-sampling task shall run with a period of 1 ± 50 ms (soft real-time).
NFR2	The safety/watchdog task shall run at the highest task priority in the system, with a period ≤ 500 ms, so faults are detected quickly regardless of what other tasks are doing.
NFR3	End-to-end latency from a sensor reading to the corresponding heater/humidifier actuation shall not exceed 2 s.
NFR4	Networking (Blynk) work shall run in its own task and shall never block, or be blocked by, the control tasks; any resource it shares with them must be mutex-protected.

ID	Requirement
NFR5	All resources shared between tasks (the latest-sensor-reading structure, the I2C bus/display, the motor) shall be protected by a mutex or accessed only through a queue — no unsynchronized shared globals.
NFR6	The system shall continue safe local operation (control loop + local alarm) for at least 24 h with no Wi-Fi connectivity.
NFR7	An egg-turning action shall complete within 10 s and shall be mutually exclusive with any operation that reads the same I2C bus/display.
NFR8	Loss or corruption of a single sensor reading shall not cause the heater or humidifier to latch permanently on.

6. System Architecture

The system is organized as three layers: (1) sensing and limit-switch inputs, (2) the ESP32 running FreeRTOS, which owns all control logic and shared state, and (3) actuators plus a local display, with the Blynk Cloud and mobile/web app providing a remote HMI over Wi-Fi. All actuator decisions are made locally on the ESP32; Blynk is a monitoring/override layer, not a dependency for safe operation (see NFR6).

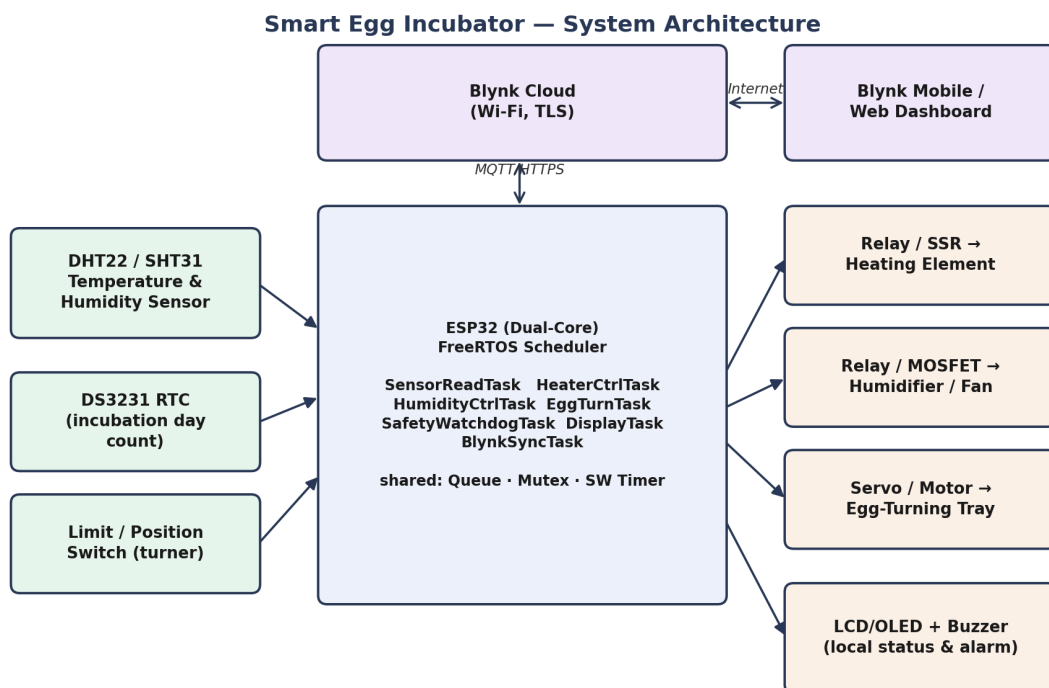


Figure 1: Smart Egg Incubator system architecture.

7. Hardware Requirements (Bill of Materials)

The list below is a suggested, budget-friendly configuration. Substitutions are allowed with instructor approval, provided the RTOS task structure in Section 9 still applies.

Component	Suggested Part	Qty	Purpose
Microcontroller	ESP32 DevKit V1 (WROOM-32)	1	Main controller; runs FreeRTOS and the Wi-Fi/Blynk stack

Component	Suggested Part	Qty	Purpose
Temp./humidity sensor	DHT22 (AM2302); SHT31 (I2C) for higher accuracy	1	Environmental sensing
Heating element	12 V PTC ceramic heating plate, or a low-wattage (25–40 W) incandescent bulb	1	Provides heat
Heater driver	5 V opto-isolated relay module or solid-state relay (SSR), rated for the load	1	Switches the heater under GPIO control
Humidifier	Small 12 V ultrasonic mist-maker/fogger disc, or a fan blowing over a water tray	1	Raises relative humidity
Humidifier driver	Relay module or NPN/MOSFET switch	1	Switches the humidifier/fan under GPIO control
Egg-turning actuator	Continuous-rotation hobby servo + tilting cradle (simplest), or 12 V geared DC motor + roller tray	1	Tilts/rotates the egg tray on schedule
Local display	16×2 or 20×4 I2C LCD, or 0.96" I2C OLED	1	Shows status without the app
Local alarm	5 V piezo buzzer	1	Audible fault/alarm indicator
Real-time clock (recommended)	DS3231 RTC module	1	Keeps the incubation day count accurate across resets/power loss
Position sensing (optional)	Micro limit switch or magnetic reed switch	1–2	Confirms the tray reached its turned position
Power supply	5 V/2 A USB supply for the ESP32; separate 12 V supply for heater/fogger as needed	1–2	Powers the system; keep control and heater supplies separate
Safety cutoff	Inline thermal fuse / cutoff switch rated below the heater's failure temperature	1	Hardware backup if the software control loop fails
Enclosure	Insulated foam or wood box with a viewing window and small ventilation holes	1	Houses the eggs and stabilizes the microclimate
Misc.	Jumper wires, breadboard/perfboard, screw terminals, egg tray/cradle hardware	—	Assembly

7.1 Safety Notes

- Prefer a 12 V heating pad/element over a mains-voltage bulb; if mains voltage is used anywhere, wiring must be fully insulated, fused, and done under instructor supervision.
- The relay/SSR driving the heater should default to OFF on power-up and on any software fault (fail-safe, not fail-on).
- Include the hardware thermal cutoff from the BOM as an independent backup — it must be able to stop the heater even if the ESP32 or its firmware misbehaves.
- Keep the humidifier/water reservoir electrically isolated from mains wiring.

8. Software and Development Tools

- Arduino-ESP32 core (or ESP-IDF) with FreeRTOS APIs (xTaskCreate, xQueueCreate, xSemaphoreCreateMutex, xTimerCreate, ...).
- Blynk library (BlynkSimpleEsp32.h) and a Blynk account/template, following the setup steps in the course's ESP32 + Blynk supporting document (template → device → datastreams → dashboard widgets).
- Sensor library for the chosen sensor (e.g., “DHT sensor library” for DHT22, or “Adafruit SHT31” for SHT31).
- Servo or motor-driver library as appropriate (e.g., ESP32Servo).
- Optional: RTCLib for the DS3231, LiquidCrystal_I2C or U8g2 for the local display.

9. FreeRTOS Task Design

The prototype must be structured as the independent tasks below (names may differ; the responsibilities and synchronization must not be collapsed into a single task or into loop()). Priorities are relative to each other on a given core; exact numeric values should be justified in the report.

Task	Responsibility	Period	Priority	Key sync objects
SensorReadTask	Reads temperature/humidity; publishes the latest reading	1 s	High	sensorQueue (send)
HeaterCtrlTask	Bang-bang/PID control of the heater relay from the latest reading	1 s	High	sensorQueue (receive), heaterMutex
HumidityCtrlTask	Controls humidifier/fan; switches setpoint at lockdown	2 s	Medium	sensorQueue (receive)
EggTurnTask	Drives the servo/motor on a timer; disabled after lockdown	SW timer (e.g., every 4 h)	Medium	turnTimer, motorMutex
SafetyWatchdogTask	Checks for out-of-range values, sensor faults, and stuck actuators; raises alarms	500 ms	Highest	sensorQueue (peek), alarmSemaphore
DisplayTask	Updates the local LCD/OLED with status	1 s	Low	displayMutex
BlynkSyncTask	Maintains Wi-Fi/Blynk connection; publishes telemetry; handles remote commands	runs Blynk.run() continuously; publishes every 2 s	Medium	sensorQueue (peek), displayMutex

9.1 Inter-Task Communication and Synchronization

- Queue (sensorQueue): SensorReadTask is the single producer of a small SensorData_t{temperature, humidity, timestamp} struct; HeaterCtrlTask, HumidityCtrlTask, SafetyWatchdogTask, DisplayTask and

BlynkSyncTask read the latest value (e.g., via xQueuePeek or a small “latest value” mailbox pattern) rather than sharing a raw global.

- Mutex (displayMutex): guards the shared I2C bus / LCD so DisplayTask and BlynkSyncTask never write to it at the same time.
- Mutex (motorMutex): ensures the turning motor and any task probing the limit switch don’t run concurrently (supports NFR7).
- Software timer (turnTimer): fires the periodic egg-turn instead of a task busy-polling a countdown — the timer callback either performs the turn directly or notifies EggTurnTask.
- Semaphore (alarmSemaphore): given by SafetyWatchdogTask and taken by a lightweight alarm/notify path, decoupling fault detection from the (potentially slower) act of sending a Blynk notification.
- Task notifications may be used instead of a semaphore for simple one-to-one signalling (e.g., an ISR on the limit switch waking EggTurnTask).

10. Blynk Dashboard Design

Following the template/device/datastream workflow from the course’s supporting document, create one template (e.g., “SmartIncubator”) with the datastreams and dashboard widgets below. Virtual pin numbers are suggestions.

Widget	V-Pin	Function
Gauge	V0	Live temperature (°C)
Gauge	V1	Live relative humidity (%)
LED	V2	Heater ON/OFF indicator
LED	V3	Humidifier ON/OFF indicator
LED	V4	Egg-turning motor active indicator
Labeled Value	V5	Incubation day counter (“Day x of 21”)
Switch	V6	Manual / Auto mode toggle
Slider	V7	Manual setpoint override (test mode only)
Button	V8	Manual egg-turn trigger (test mode only)
Notifications + Terminal	V9	Alarm/event log (out-of-range, sensor fault, turn completed)

11. Safety and Fault Handling

- Independent hardware cutoff: a thermal fuse/cutoff in series with the heater that trips regardless of firmware state (see Section 7.1).
- Fail-safe defaults: heater and humidifier relays default OFF at boot and whenever SafetyWatchdogTask cannot confirm a valid recent sensor reading.
- Sensor-fault detection: treat NaN, out-of-physical-range, or stale (not updated within N seconds) readings as a fault, not as “temperature = 0”.
- Grace period before alarm: avoid nuisance alarms from a single noisy reading by requiring the out-of-range condition to persist (default 2 minutes, FR7) before alerting.
- Graceful degradation: local control and local alarm must keep working with no Wi-Fi (NFR6); Blynk connectivity is an enhancement, not a dependency.

- Watchdog task priority: SafetyWatchdogTask runs at the highest priority precisely so a runaway or blocked lower-priority task cannot delay fault detection.

12. Suggested Timeline

This is a compressed 15-day sprint schedule, not the generic multi-week pace of a self-paced lab project. It only holds under three conditions: (1) every BOM item in Section 7 is ordered or already in hand before Day 1 — procurement, not coding, is the single biggest risk to this schedule; (2) scope is capped at FR1–FR10 and the 7-task RTOS structure in Section 9, with the Section 16 extensions explicitly out of scope; and (3) validation uses the calibration-run path in Section 15 only — a full 21-day fertilized-egg hatch cannot fit in 15 days regardless of team effort.

Day(s)	Milestone
1–2	Kickoff: requirements locked to FR1–FR10, BOM ordered/confirmed in hand, roles assigned, architecture diagram drafted.
3–5	Hardware bring-up: ESP32 + sensor reading over Serial; heater relay, humidifier, and turning actuator bench-tested individually (not yet integrated).
6–9	FreeRTOS core: task/queue/mutex/software-timer skeleton built and verified with Serial logging (no cloud yet); heater and humidity control loops implemented.
10–12	Blynk integration and safety layer in parallel: template/datastreams/dashboard widgets, BlynkSyncTask, manual/test mode; SafetyWatchdogTask, fail-safe defaults, local alarm.
13–14	Full integration into the enclosure; setpoint calibration run with a data logger; bug-fixing and, only if time remains, one Section 16 extension.
15	Final report and live demonstration.

13. Deliverables

- A working hardware prototype (incubator enclosure with all electronics installed).
- Complete, commented source code implementing the task design in Section 9.
- A wiring/connection diagram for the final build.
- A short project report: filled-in requirements, final BOM with substitutions noted, task/priority table as actually implemented, test results, and known limitations.
- Blynk dashboard (screenshots or live access) demonstrating telemetry, an alert, and a manual override.
- A live in-class demonstration.

14. Suggested Grading Rubric

Weights are a suggested starting point for the instructor and can be adjusted.

Criterion	Weight
Requirements and design documentation	15%
Hardware assembly, wiring quality, and safety practice	15%
FreeRTOS task design — correct decomposition, priorities, and synchronization	25%

Control-loop correctness (temperature, humidity, turning schedule)	15%
Blynk IoT integration and dashboard	15%
Live demo and Q&A	10%
Final report	5%

15. Expected Outcome / Prototype Description

The finished prototype is a benchtop smart incubator that autonomously holds its internal temperature within the target band and follows the two-stage humidity profile (setting → lockdown) inside an insulated enclosure, automatically tilts the egg tray on schedule and stops during lockdown, shows live status locally, and streams telemetry and alerts to a Blynk dashboard so a user can monitor — and, in test mode, override — the process remotely.

Within this 15-day schedule, validation uses the calibration-run path only: hold the setpoints on the bench with a data logger in place of eggs for as long as the schedule allows (ideally spanning the Day 13–14 integration window overnight), and demonstrate at least one full egg-turn cycle and one deliberately triggered/cleared alarm live at the Day 15 demo. A full 21-day fertilized-egg hatch does not fit this schedule under any amount of effort — it needs its own calendar time regardless of how fast the team builds. Teams who want to carry the prototype through an actual hatch should treat that as optional follow-on work after the graded deliverable, subject to instructor approval and any applicable animal-care guidance.

16. Optional Extensions

Out of scope for the 15-day schedule in Section 12 — attempting any of these will crowd out the time FR1–FR10 and the safety layer need. Treat these as follow-on ideas only, for a team that finishes early or continues the project after grading:

- Replace bang-bang heater control with a PID loop for tighter temperature regulation.
- Add an ESP32-CAM for periodic “candling” photos, viewable from the Blynk app.
- Log historical data to an SD card or the Blynk cloud for a full-incubation graph.
- Support multiple species/presets (chicken/duck/quail) selectable from the app, each with its own setpoint and duration.
- Add a small battery/solar backup to keep the safety watchdog and local alarm alive through short power outages.

17. References

[1] Course supporting document: “ESP32 and Blynk IoT Platform — An Overview of Features, Applications, and Implementation Techniques,” ENC55140 Lab, Birzeit University.

[2] Mississippi State University Extension Service, “Important Incubation Factors” — temperature, humidity, and turning guidance for chicken eggs.

[3] Purina Animal Nutrition, “A 21-Day Incubation Guide to Hatch Eggs at Home.”

[4] Espressif Systems, ESP32 documentation; FreeRTOS API reference (freertos.org).

[5] Blynk documentation — docs.blynk.io.

Appendix A. Suggested FreeRTOS + Blynk Code Skeleton

This is a starting skeleton, not a full solution — sensor/actuator driver calls, PID gains, and Blynk virtual-pin handlers must be completed by the team. It shows the expected task/queue/mutex/timer structure from Section 9.

```
#define BLYNK_TEMPLATE_ID      "TMPLxxxxxxx"
#define BLYNK_TEMPLATE_NAME    "SmartIncubator"
#define BLYNK_AUTH_TOKEN       "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

// ---- Wi-Fi / Blynk ----
char ssid[] = "YOUR_SSID";
char pass[] = "YOUR_PASSWORD";

// ---- Pins ----
#define DHT_PIN      4
#define HEATER_PIN   16
#define HUMID_PIN    17
#define SERVO_PIN    18
#define BUZZER_PIN   19
DHT dht(DHT_PIN, DHT22);

// ---- Shared data & RTOS objects ----
struct SensorData_t { float temperature; float humidity; uint32_t ts; bool valid; };
QueueHandle_t  sensorQueue;      // holds the latest SensorData_t (length 1, overwrite)
SemaphoreHandle_t displayMutex;   // guards the I2C bus / LCD
SemaphoreHandle_t motorMutex;     // guards the turning motor
SemaphoreHandle_t alarmSemaphore; // given by watchdog, taken by alarm/notify path
TimerHandle_t  turnTimer;        // periodic egg-turn trigger
volatile int    incubationDay = 0;
volatile bool    lockdown = false;
volatile bool    manualMode = false;

// ---- Task: read sensor, publish latest value ----
void SensorReadTask(void *pv) {
    for (;;) {
        SensorData_t d;
        d.temperature = dht.readTemperature();
        d.humidity    = dht.readHumidity();
        d.ts          = millis();
        d.valid = !isnan(d.temperature) && !isnan(d.humidity);
        xQueueOverwrite(sensorQueue, &d);      // always keep only the latest reading
        vTaskDelay(pdMS_TO_TICKS(1000));        // NFR1: 1 s period
    }
}

// ---- Task: heater bang-bang/PID control ----
void HeaterCtrlTask(void *pv) {
    SensorData_t d;
    for (;;) {
        if (xQueuePeek(sensorQueue, &d, 0) == pdTRUE && d.valid) {
            bool heaterOn = d.temperature < 37.2; // TODO: replace with PID / hysteresis band
            digitalWrite(HEATER_PIN, heaterOn ? HIGH : LOW);
            Blynk.virtualWrite(V2, heaterOn);
        } else {
            digitalWrite(HEATER_PIN, LOW); // fail-safe: no valid reading -> heater OFF
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// ---- Task: humidity control (setting vs. lockdown target) ----
void HumidityCtrlTask(void *pv) {
    SensorData_t d;
    for (;;) {
        if (xQueuePeek(sensorQueue, &d, 0) == pdTRUE && d.valid) {
            float target = lockdown ? 67.0 : 52.0; // TODO: tune per Section 3 setpoints
            bool humidOn = d.humidity < target;
            digitalWrite(HUMID_PIN, humidOn ? HIGH : LOW);
            Blynk.virtualWrite(V3, humidOn);
        }
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}
```

```

    }
}

// ---- Software timer callback: trigger a turn every N hours ----
void turnTimerCallback(TimerHandle_t xTimer) {
    if (!lockdown) xTaskNotifyGive(eggTurnTaskHandle); // wake EggTurnTask
}

// ---- Task: perform the physical egg-turn ----
TaskHandle_t eggTurnTaskHandle;
void EggTurnTask(void *pv) {
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY); // wait for timer/manual trigger
        if (xSemaphoreTake(motorMutex, pdMS_TO_TICKS(500)) == pdTRUE) {
            // TODO: drive servo/motor to tilt tray, confirm with limit switch, NFR7: <=10 s
            Blynk.virtualWrite(V4, 1);
            vTaskDelay(pdMS_TO_TICKS(3000)); // placeholder for actual turn motion
            Blynk.virtualWrite(V4, 0);
            xSemaphoreGive(motorMutex);
        }
    }
}

// ---- Task: highest-priority safety watchdog ----
void SafetyWatchdogTask(void *pv) {
    SensorData_t d;
    for (;;) {
        bool fault = (xQueuePeek(sensorQueue, &d, 0) != pdTRUE) || !d.valid
                    || d.temperature < 30 || d.temperature > 42
                    || d.humidity < 20 || d.humidity > 90;

        if (fault) {
            digitalWrite(HEATER_PIN, LOW); // fail-safe
            digitalWrite(BUZZER_PIN, HIGH);
            xSemaphoreGive(alarmSemaphore); // let the notify path handle Blynk alert
        } else {
            digitalWrite(BUZZER_PIN, LOW);
        }
        vTaskDelay(pdMS_TO_TICKS(500)); // NFR2: <=500 ms
    }
}

// ---- Task: local LCD/OLED status ----
void DisplayTask(void *pv) {
    SensorData_t d;
    for (;;) {
        if (xSemaphoreTake(displayMutex, pdMS_TO_TICKS(200)) == pdTRUE) {
            xQueuePeek(sensorQueue, &d, 0);
            // TODO: lcd.print(...) temperature, humidity, day counter
            xSemaphoreGive(displayMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// ---- Task: Blynk connection + periodic publish ----
void BlynkSyncTask(void *pv) {
    SensorData_t d;
    uint32_t lastPublish = 0;
    for (;;) {
        Blynk.run();
        if (millis() - lastPublish > 2000) { // FR6: every <=2 s
            if (xQueuePeek(sensorQueue, &d, 0) == pdTRUE && d.valid) {
                Blynk.virtualWrite(V0, d.temperature);
                Blynk.virtualWrite(V1, d.humidity);
                Blynk.virtualWrite(V5, incubationDay);
            }
            lastPublish = millis();
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

// ---- Blynk virtual-pin handlers (manual/test mode) ----
BLYNK_WRITE(V6) { manualMode = param.asInt(); }
BLYNK_WRITE(V8) { if (manualMode) xTaskNotifyGive(eggTurnTaskHandle); }

void setup() {
    Serial.begin(115200);
}

```

```

pinMode(HEATER_PIN, OUTPUT); pinMode(HUMID_PIN, OUTPUT); pinMode(BUZZER_PIN, OUTPUT);
dht.begin();
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

sensorQueue    = xQueueCreate(1, sizeof(SensorData_t));
displayMutex   = xSemaphoreCreateMutex();
motorMutex     = xSemaphoreCreateMutex();
alarmSemaphore = xSemaphoreCreateBinary();

xTaskCreatePinnedToCore(SensorReadTask,    "Sensor", 4096, NULL, 3, NULL, 1);
xTaskCreatePinnedToCore(HeaterCtrlTask,    "Heater", 2048, NULL, 3, NULL, 1);
xTaskCreatePinnedToCore(HumidityCtrlTask,  "Humid",  2048, NULL, 2, NULL, 1);
xTaskCreatePinnedToCore(EggTurnTask,       "Turn",   2048, NULL, 2, &eggTurnTaskHandle,
1);
xTaskCreatePinnedToCore(SafetyWatchdogTask, "Watchdog", 2048, NULL, 4, NULL, 1);
xTaskCreatePinnedToCore(DisplayTask,       "Display", 2048, NULL, 1, NULL, 1);
xTaskCreatePinnedToCore(BlynkSyncTask,     "Blynk",   4096, NULL, 2, NULL, 0);

turnTimer = xTimerCreate("TurnTimer", pdMS_TO_TICKS(4UL * 60 * 60 * 1000),
                        pdTRUE, NULL, turnTimerCallback); // every 4 h, TODO tune
xTimerStart(turnTimer, 0);
}

void loop() {
    // Intentionally empty: all work happens in FreeRTOS tasks above.
    vTaskDelay(pdMS_TO_TICKS(1000));
}

```

Appendix B. Egg Incubation Parameter Reference

Chicken-egg figures below are drawn from poultry-extension references [2],[3]. Duck/quail/goose figures are typical published ranges and should be re-verified before use if a team targets one of those species.

Species	Temp. (°C)	Humidity (setting → lockdown)	Turns/day	Duration
Chicken	37.2–37.8	50–55% → 65–70% (last 3 days)	3–5	≈ 21 days
Duck	37.2–37.5	55–60% → 65–75%	3–5	≈ 28 days
Quail	37.5–37.8	45–50% → 65–70%	3–5	≈ 17–18 days
Goose	37.2–37.5	55–65% → 70–80%	3–5	≈ 28–35 days